

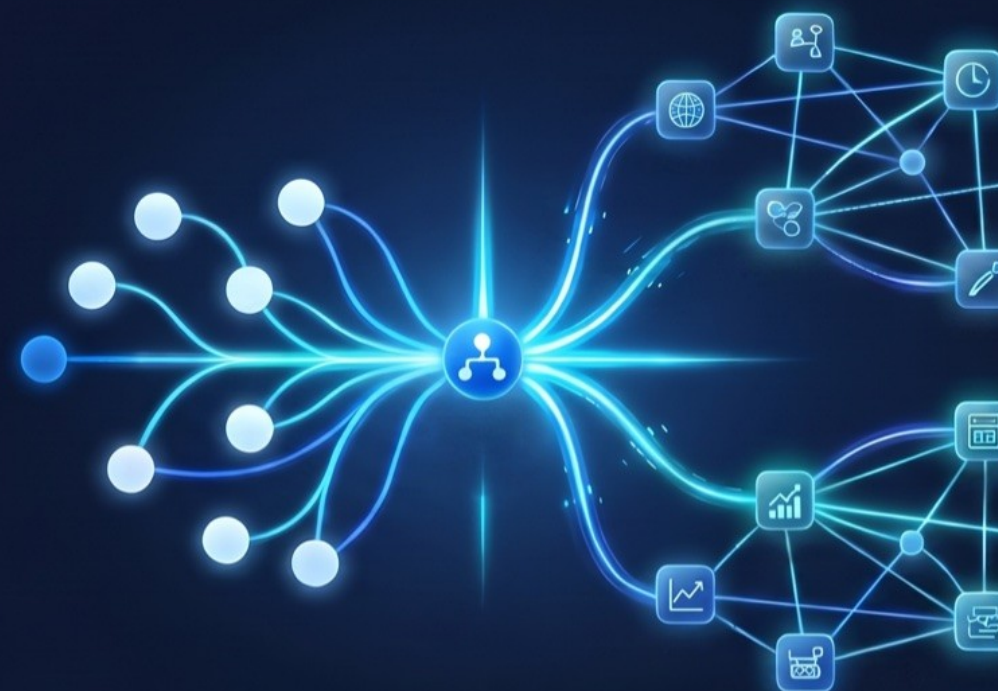
EPISÓDIO 3

# MetricFlow local: métricas como código

Defina, valide, compile e execute sem conta  
dbt Platform

---

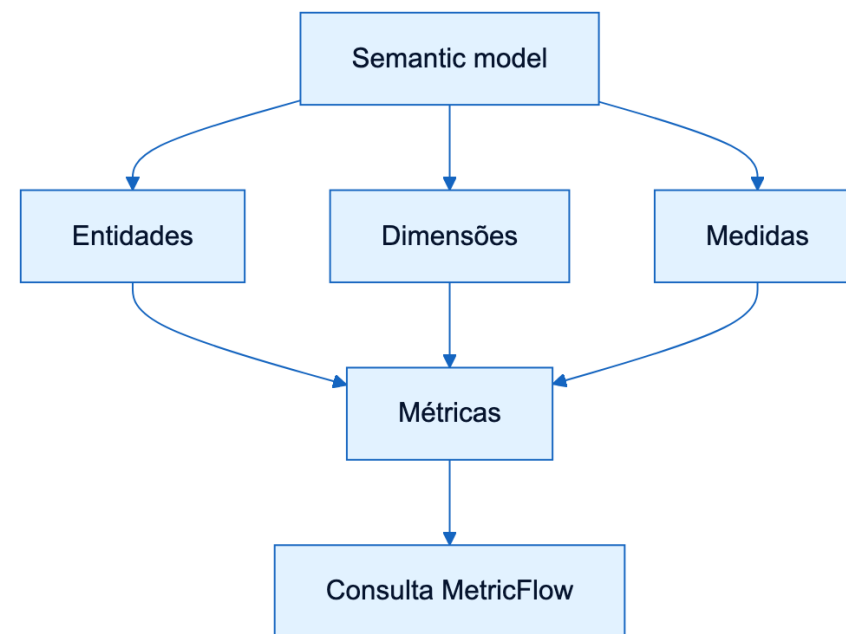
Checkpoint 03 • DuckDB local • sem cloud



# Uma métrica declarada uma vez pode gerar o mesmo SQL para todos os consumidores.

---

A pergunta que guia a aula



# Ao final, você consegue...

**01**

**Declarar entidades e  
dimensões**

**02**

**Criar métricas  
simples e razão**

**03**

**Reconciliar SQL com  
ground truth**

# MetricFlow compila; não adivinha

A força vem de definições explícitas, não de geração probabilística.

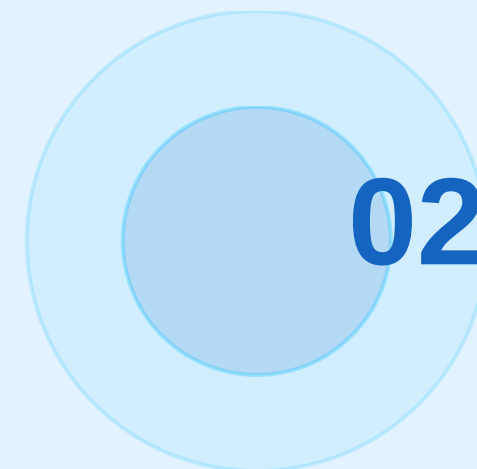
- Consulta lógica
- Grafo semântico
- SQL executável



# Entidade começa pelo grão

`order_id` só pode ser primary se o modelo tiver uma linha por pedido.

- Primary: order
- Foreign: customer
- Unicidade testada antes da métrica



# Entidade começa pelo grão

## Entidade

- Primary: order
- Foreign: customer

## Grão físico

- Unicidade testada antes da métrica

# O exemplo oficial conecta schema e métricas

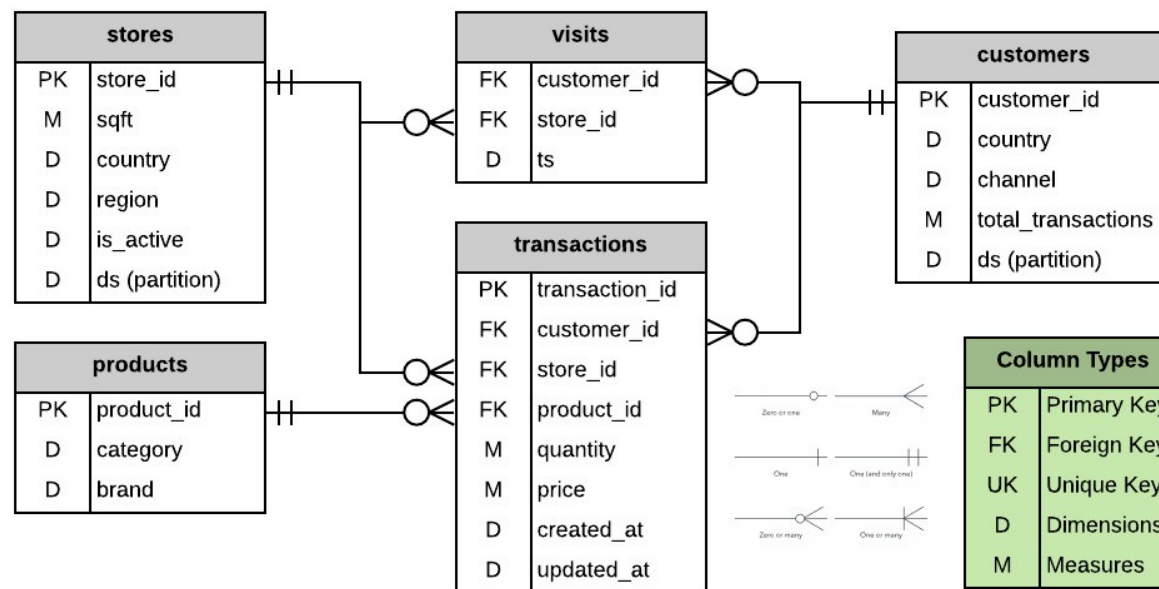
Chaves e tipos de coluna sustentam os caminhos do grafo.

- PK e FK
- Dimensões
- Medidas



# O exemplo oficial conecta schema e métricas

Chaves e tipos de coluna sustentam os caminhos do grafo.



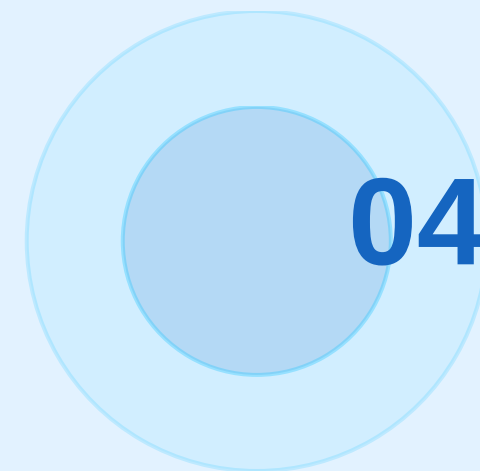
Exemplo oficial dbt Labs — Apache 2.0



# Tempo também é contrato

A dimensão padrão precisa refletir a pergunta de negócio.

- order\_purchase\_date
- Granularidade diária
- Time spine para operações temporais



# Tempo também é contrato

01

Evento



02

Grão



03

Calendário



04

Métrica

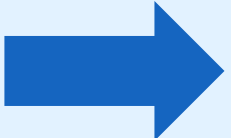
# Métricas simples vivem perto do dado

Contagem, soma e receita entregue reutilizam colunas já testadas.

- orders: count\_distinct
- gross\_revenue: sum
- delivered\_revenue: sum



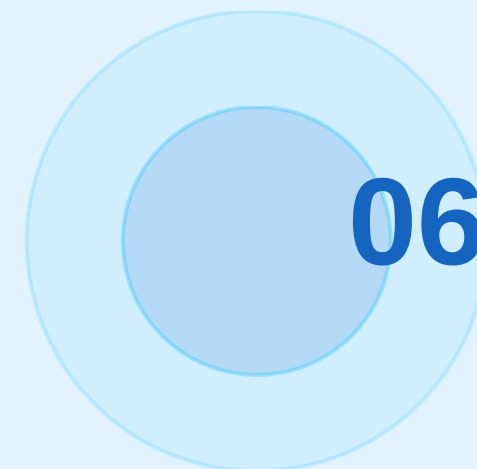
## Métricas simples vivem perto do dado

**8 pedidos × 740 bruto**  **630 entregue**

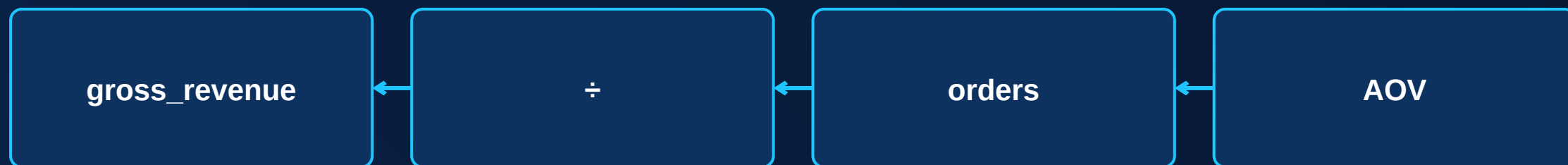
# Razões reutilizam métricas nomeadas

Ticket médio compõe gross\_revenue e orders sem copiar a fórmula.

- Numerador governado
- Denominador governado
- Precisão e zero documentados



# Razões reutilizam métricas nomeadas



# Demonstração — execute

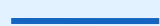
```
python course.py checkpoint 03
```

Copie exatamente como aparece. O launcher escolhe o ambiente correto.

# Demonstração — observe

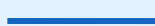
**01**

**mf validate-configs  
sem erro**



**02**

**Consulta mensal  
compilada**



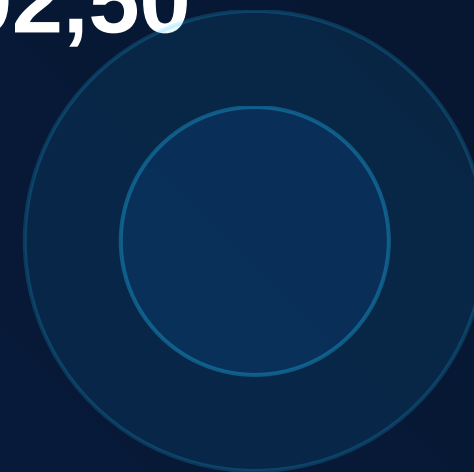
**03**

**Janeiro a abril  
reconciliados**



RESULTADO ESPERADO

**8 pedidos - 740 bruto - 630 entregue - 92,50  
AOV**



Checkpoint 03 • evidência reproduzível

# O mesmo recurso afeta três decisões

**01**

**Governança: Git e ownership**

—

**02**

**FinOps: reuso com cardinalidade**

—

**03**

**IA: nomes em vez de joins inventados**

**2 → 13**

**contribuidores reportados pela Inventa**

O laboratório comprova apenas validação, SQL e reconciliação.

# Limitações que precisam permanecer visíveis

- MetricFlow não decide a definição da métrica
- Alta cardinalidade ainda custa
- Biblioteca aberta não é o serviço hospedado

## RADAR

**A matriz de suporte do Core 2.0/Fusion não é assumida igual ao Core 1.12.**



Novidade observada  $\neq$  recomendação de produção

## PRÓXIMO PASSO

# Implemente uma métrica simples e uma razão e reconcilie os valores.

Checkpoint 03 • roteiro completo no repositório